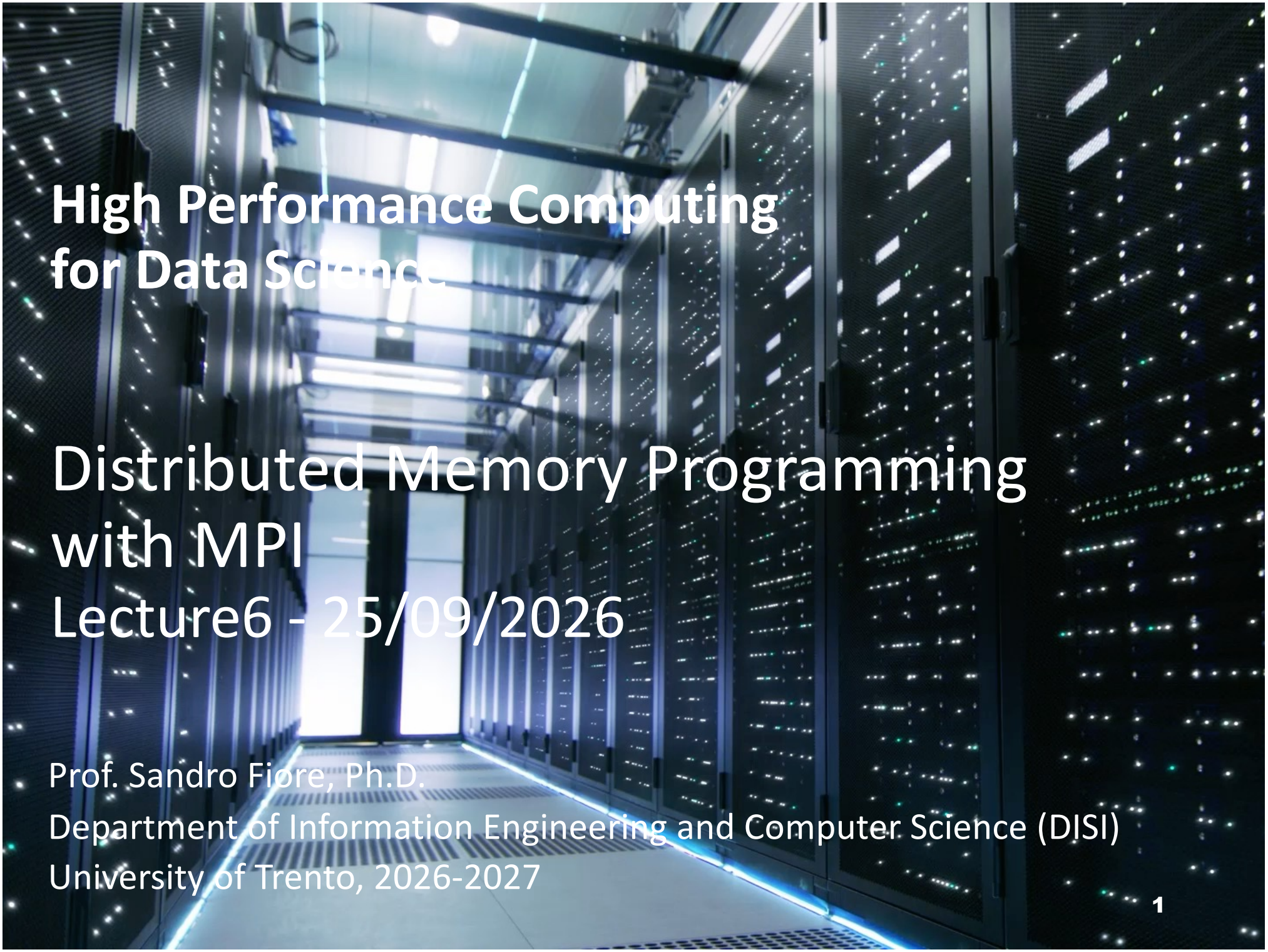


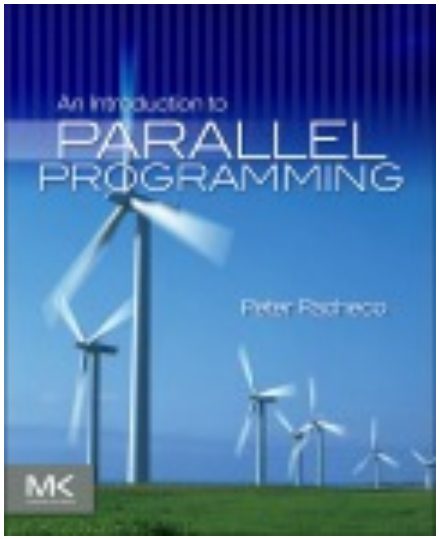
High Performance Computing for Data Science

Distributed Memory Programming with MPI
Lecture6 - 25/09/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Today's outline and material

- Introduction
- Communication
- Hands-on



Today's material

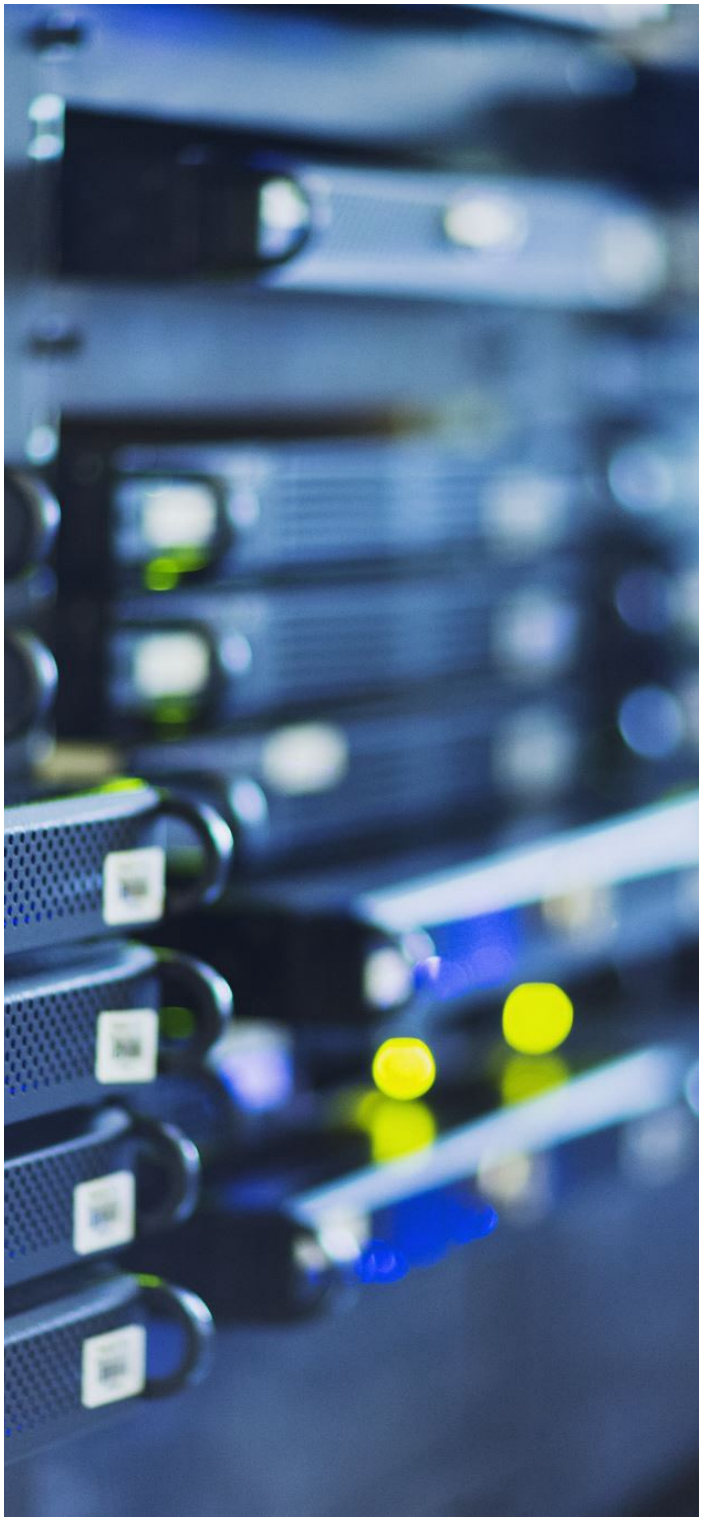
Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 3
Distributed Memory Programming with MPI

A few info before we start

- Account request: please proceed if you haven't done it so far
- Project teams (today)
- No lectures next week (Oct 1 and Oct 2)
- October 9 (or 16) visit to the HPC Center in Povo2

- First speakers confirmed (afternoon seminars)
  - *Wes Brewer (Oak Ridge National Lab)*
    - *https://www.ornl.gov/staff-profile/wes-h-brewer*
  - *Valentine Anantharaj (Oak Ridge National Lab)*
    - *https://www.ornl.gov/staff-profile/valentine-g-anantharaj*



Virtual Private Network @ UniTrento

- Access to the HPC cluster @ UniTrento is provided via Virtual Private Network (VPN)

Info about VPN@UniTrento:

VPN

- https://servicedesk.unitn.it/goauth/en?s=S00180

MFA idp

- https://servicedesk.unitn.it/goauth/en?kb=KB0011764

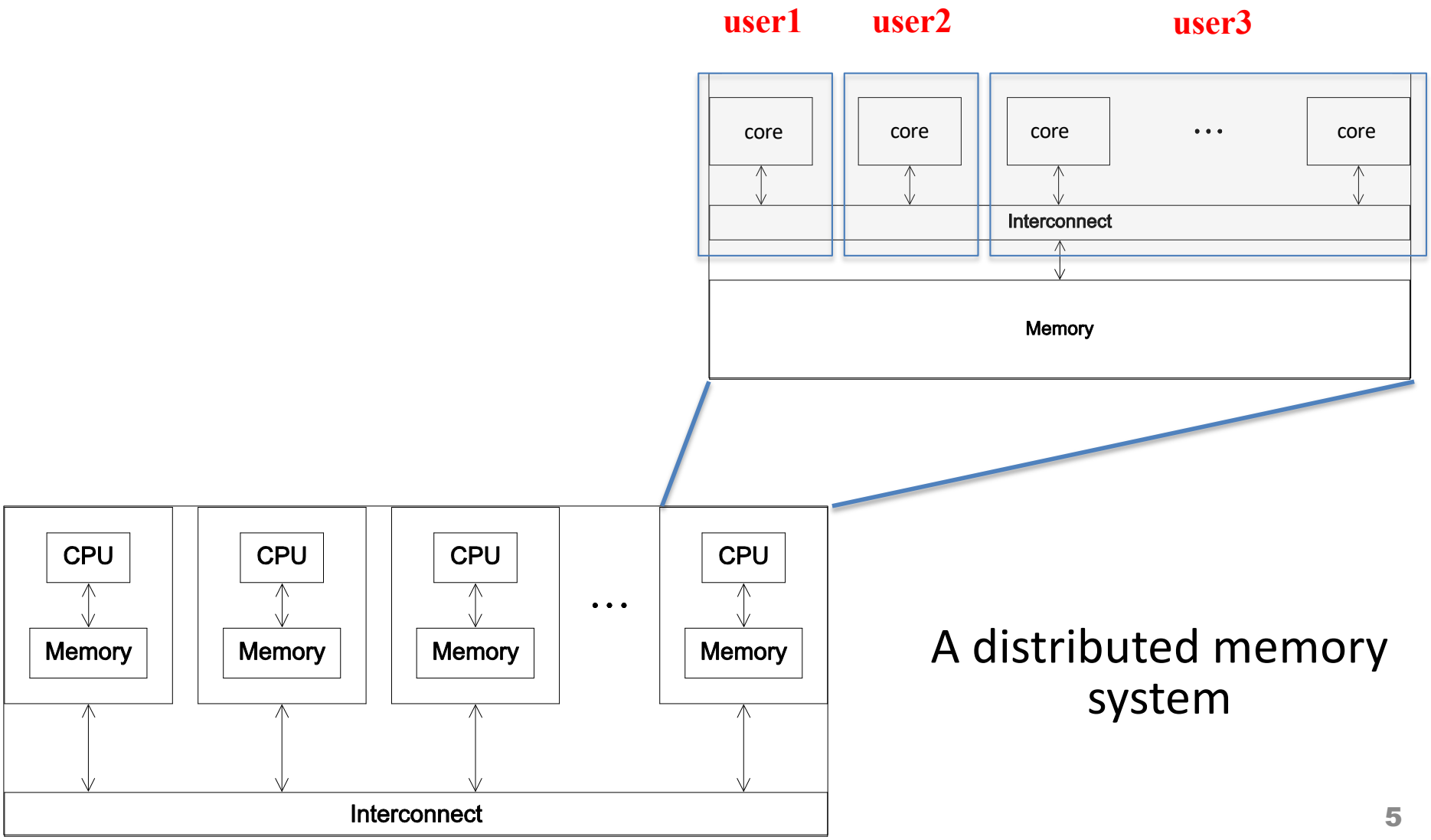
Login Node

- https://servicedesk.unitn.it/goauth/en?kb=KB0013882

Cluster Policy

- https://servicedesk.unitn.it/goauth/en?kb=KB0010398

Be aware other people are using the HPC cluster at the same time



A distributed memory system

Unitn HPC cluster usage conditions

English

3. Home directory quota check

3.1 Every week a check of the used space is perfor
exceeded.

3.2 In the latter case, the user must promptly reduc
reasoned request to benefit, for a limited period of
may be granted, denied or withdrawn based on the

3.3 After 4 unsuccesful warnings, non-compliant us
home directory will still be possible via FTPs servic
or remove their data.

4. Home directory of users with expired accounts

4.1 Users must remove all data from their home dir
cluster expires. Access to the home directory by us
via FTPs service (here the instructions).

4.2 On a weekly basis, a check is carried out that id
whose access to the cluster has expired, and an em
data that the user want to keep via FTPs and to ren

4.3 To safeguard the resources available on the syst
home will be removed **without any possibility of re**

5. Emergency

In the event of exceptional situations, when the infrastructure risks to stop operations due to the exhaustion of available space on the volume that contains the users' homes, after discussion with the Rector's Delegate for Services and IT technologies, the following extraordinary measures can be adopted:

5.1 It will be possible to intervene, even without the approval of the interested parties, on the home directories of users who have exceeded the quota granted, also through their removal.

5.2 It will be possible to intervene on the home directories of users whose access to the cluster has expired, also by deleting them before the period indicated in point 4.3.

6. Prohibitions

6.1 The use of HPC resources for activities other than teaching and research is strictly prohibited.

6.2 Calculations, simulations, etc. can only be started via the **qsub** command. In particular:

- It is forbidden to execute any job on the head-nodes of the cluster (hpc-head-n1 and hpc-head-n2);
- it is forbidden to access the compute nodes via ssh.

6.3 The jobs submitted by each single user are visible by all users of the cluster; in addition, some information relating to the jobs (name, date and time of submission and execution, resources requested, etc.), are collected and used by the cluster accounting system for statistical purposes and for reporting. For these reasons, in order to protect privacy, job names must in no way contain information that can be traced back to people. In cases where this may occur, pseudo anonymization techniques of the job names must be used (e.g. genetic_test_john_brown.sh can become genetic_test_user_4451.sh).

6.4 Failure to comply with the prohibitions 6.1, 6.2 and 6.3, barring cases whose gravity is such as to warrant other measures (disciplinary of the University or reporting to the authorities), provide for:

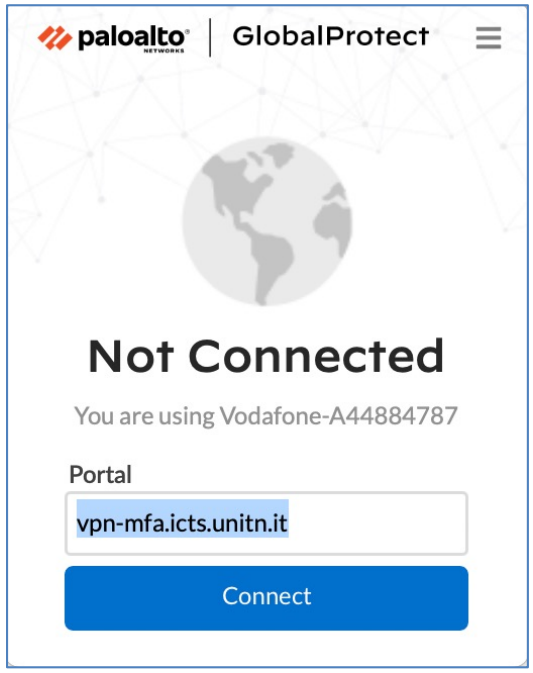
- at the first occurrence, an informal warning by email;
- in case of reiteration after the first warning, the case will be presented to the Rector's Delegate for IT Services and Technologies, who will arrange the measures to be taken, up to the prohibition of accessing the cluster.

https://servicedesk.unitn.it/goauth/en?kb=KB0010398

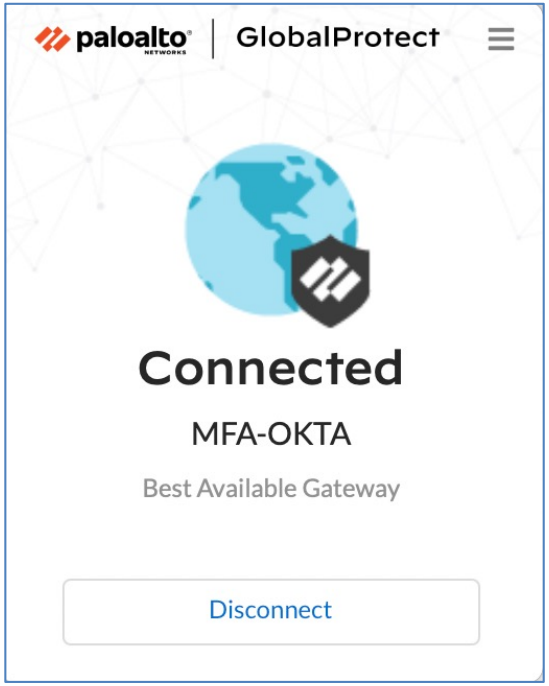
Virtual Private Network @ UniTrento

1) By installing the Global Protect VPN on your machine:

2) and signing in (using UNITN Credentials)



Google AuthN



3) you will get connected to the VPN

Accessing the HPC Cluster

Accessing the HPC Cluster

To access the University HPC cluster (HPC3), connect via SSH to the following address:

- hpc3-login.unitn.it

Linux and macOS

From a terminal, run the following command:

- ssh university_username@hpc3-login.unitn.it

Windows

Use an SSH client such as **PuTTY** or **MobaXTerm**. Enter your University username as the **username**, your University Services password as the **password**, and hpc3-login.unitn.it as the **host address**.

VPN

If you are connecting from outside the University network, you must activate the VPN service before connecting to the cluster.

PBS User Guide



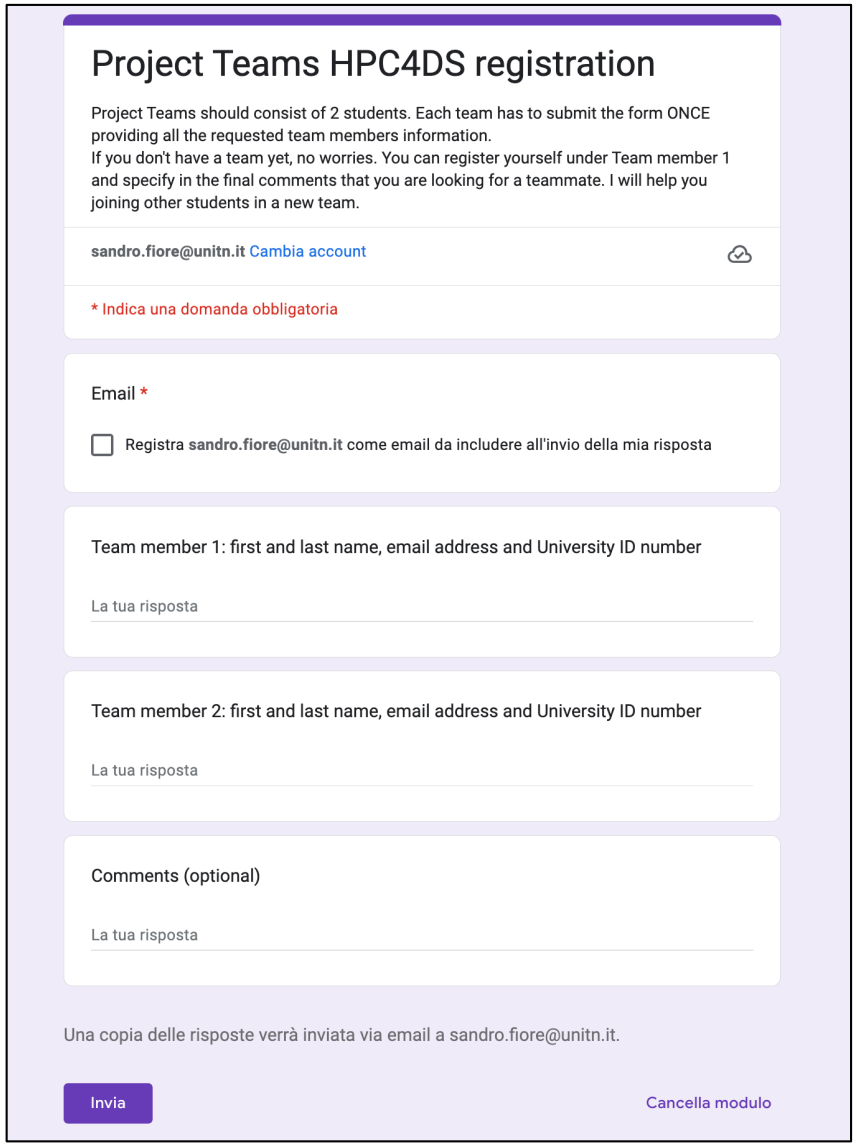
ALTAIR

ONLY FORWARD

Altair® PBS Professional® 2025.2.0

Pdf on Moodle

Registration form Project Teams



Project Teams HPC4DS registration

Project Teams should consist of 2 students. Each team has to submit the form ONCE providing all the requested team members information.
If you don't have a team yet, no worries. You can register yourself under Team member 1 and specify in the final comments that you are looking for a teammate. I will help you joining other students in a new team.

sandro.fiore@unitn.it Cambia account

* Indica una domanda obbligatoria

Email *

Registra sandro.fiore@unitn.it come email da includere all'invio della mia risposta

Team member 1: first and last name, email address and University ID number

La tua risposta

Team member 2: first and last name, email address and University ID number

La tua risposta

Comments (optional)

La tua risposta

Una copia delle risposte verrà inviata via email a sandro.fiore@unitn.it.

Invia

Cancella modulo

- Project Teams should consist of 2 students.
- Each team has to submit the form ONCE providing all the requested team members information.
- If you don't have a team yet, no worries. You can register yourself under Team member 1 and specify in the final comments you are looking for a teammate (I will help you joining other students in a new team).
- Link to the form: https://docs.google.com/forms/d/e/1FAIpQLSePPspo5Z_NAMOCS06HNhYPYRRUmrwp9KUCa7q5xN5dZ5KPkg/viewform

Visit to the Data Center

- October, Friday 9 or Friday 16

Hello world MPI program

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // Initialize the MPI environment. The two arguments to MPI Init are not
  // currently used by MPI implementations, but are there in case future
  // implementations might need the arguments.
  MPI_Init(NULL, NULL);

  // Get the number of processes
  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  // Get the rank of the process
  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  // Print off a hello world message
  printf("Hello world from process rank %d out of %d processors\n", world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

MPI_Init() vs main()

```
int MPI_Init(
      int*      argc_p  /* in/out */,
      char***   argv_p  /* in/out */);
```

main(int argc, char **argv)

MPI_Init(int *argc, char ***argv)

Exercise:

> mpirun -np 4 ./hello_world 3 10

What can we say about argc and argv ?

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Init() vs main()

mpirun -np 4 ./hello_world 3 10

argc ?

process 0: ./hello_world 3 10

process 1: ./hello_world 3 10

process 2: ./hello_world 3 10

process 3: ./hello_world 3 10

argc?

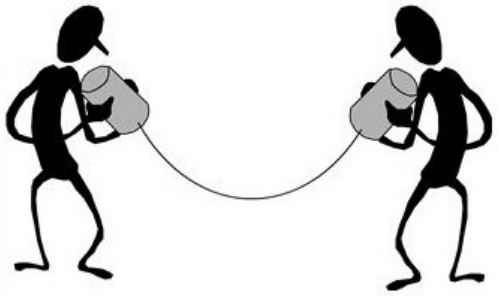
Communicators

- A collection of processes that can send messages to each other.
- MPI_Init defines a communicator that consists of all the processes created when the program is started.
- Called MPI_COMM_WORLD.

Copyright © 2010, Elsevier Inc. All rights Reserved

Communication

```
int MPI_Send(
   void*          msg_buf_p      /* in */,
   int            msg_size       /* in */,
   MPI_Datatype   msg_type       /* in */,
   int            dest           /* in */,
   int            tag            /* in */,
   MPI_Comm       communicator   /* in */);
```



MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

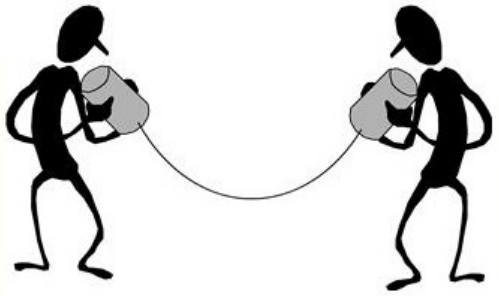
Data types

MPI datatype	C datatype
MPI_CHAR	signed **char**
MPI_SHORT	signed **short int**
MPI_INT	signed **int**
MPI_LONG	signed **long int**
MPI_LONG_LONG	signed **long long int**
MPI_UNSIGNED_CHAR	**unsigned char**
MPI_UNSIGNED_SHORT	**unsigned short int**
MPI_UNSIGNED	**unsigned int**
MPI_UNSIGNED_LONG	**unsigned long int**
MPI_FLOAT	**float**
MPI_DOUBLE	**double**
MPI_LONG_DOUBLE	**long double**
MPI_BYTE	
MPI_PACKED	

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Communication

```
int MPI_Recv(
      void*          msg_buf_p      /* out */,
      int            buf_size       /* in  */,
      MPI_Datatype   buf_type       /* in  */,
      int            source         /* in  */,
      int            tag            /* in  */,
      MPI_Comm       communicator   /* in  */,
      MPI_Status*    status_p       /* out */);
```



Copyright © 2010, Elsevier Inc. All rights Reserved

Message matching

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag,
         send_comm);
```

r

MPI_Send

src = q



MPI_Recv

dest = r

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```

q

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Receiving messages

- A receiver can get a message without knowing:
  - the amount of data in the message,
  - the sender of the message,
  - or the tag of the message.



MPI_ANY_SOURCE and MPI_ANY_TAG

Copyright © 2010, Elsevier Inc. All rights Reserved

Let's move back to our MPI program

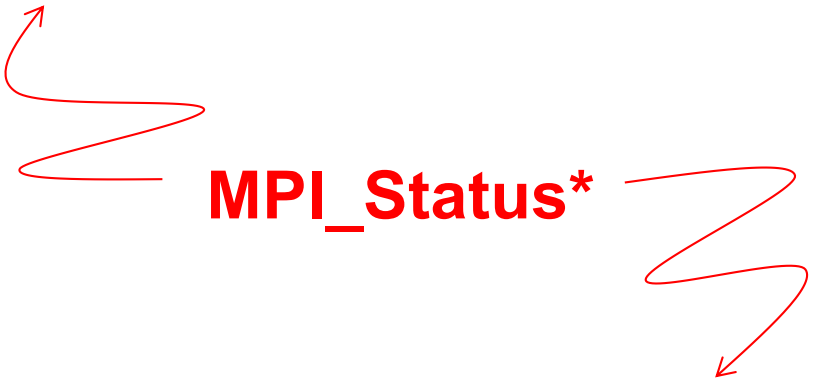
```
1  #include <stdio.h>
2  #include <string.h>  /* For strlen             */
3  #include <mpi.h>     /* For MPI functions, etc */
4
5  const int MAX_STRING = 100;
6
7  int main(void) {
8     char      greeting[MAX_STRING];
9     int       comm_sz;  /* Number of processes */
10    int       my_rank;  /* My process rank     */
11
12    MPI_Init(NULL, NULL);
13    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15
16    if (my_rank != 0) {
17       sprintf(greeting, "Greetings from process %d of %d!",
18             my_rank, comm_sz);
19       MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
20             MPI_COMM_WORLD);
21    } else {
22       printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
23       for (int q = 1; q < comm_sz; q++) {
24          MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
25             0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26          printf("%s\n", greeting);
27       }
28    }
29
30    MPI_Finalize();
31    return 0;
32 }  /* main */
```



Copyright © 2010, Elsevier Inc. All rights Reserved

status_p argument

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```



MPI_Status* status;

status.MPI_SOURCE

status.MPI_TAG

MPI_SOURCE
MPI_TAG
MPI_ERROR

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

How much data am I receiving? Bytes or objects?

```
int MPI_Get_count(
      MPI_Status*   status_p   /* in  */,
      MPI_Datatype  type       /* in  */,
      int*          count_p    /* out */);
```



Number of elements of type datatype contained in the received message.

MPI_Get_count(&status, MPI_INT, &count);

MPI_Get_count(&status, MPI_BYTE, &count);

Copyright © 2010, Elsevier Inc. All rights Reserved

Final considerations

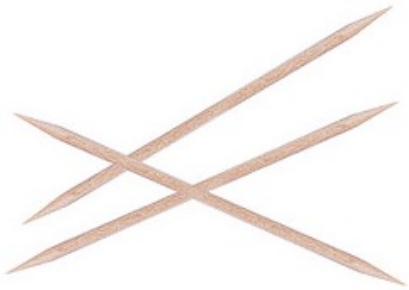
- MPI requires that messages be ***nonovertaking***
- Potential Pitfalls in the send/receive
  - For each send we must have a *corresponding* receive
  - Tags too!
  - Destination and source processes to be properly set in the send/receive
  - Otherwise, we might have hanging processes!

Copyright © 2010, Elsevier Inc. All rights Reserved

A bit on I/O

```
Proc 0 of 6 > Does anyone have a toothpick?
Proc 1 of 6 > Does anyone have a toothpick?
Proc 2 of 6 > Does anyone have a toothpick?
Proc 4 of 6 > Does anyone have a toothpick?
Proc 3 of 6 > Does anyone have a toothpick?
Proc 5 of 6 > Does anyone have a toothpick?
```

unpredictable output
(nondeterminism)



Copyright © 2010, Elsevier Inc. All rights Reserved

Input

- Most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin.
- Process 0 must read the data (scanf) and send to the other processes.

```
. . .
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
. . .
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Copying a file from your local machine to the cluster

You can use the scp command:

Syntax:

```
scp yourfile your_account@login_node:your_home_directory
```

Example:

```
scp readme.txt sandro.fiore@hpc3-login.unitn.it:/home/sandro.fiore
```

A video will be made available on moodle about scp + tar commands

Useful tools: git

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

You might find convenient to host your hands-on and project code on a github repo

https://git-scm.com/

https://git-scm.com/book/en/v2

https://github.com/

Login to the HPC cluster

- Turn on VPN
- Check with ping if the login node is reachable

```
sfiore$ ping hpc2.unitn.it
PING hpc2.unitn.it (192.168.115.242): 56 data bytes
64 bytes from 192.168.115.242: icmp_seq=0 ttl=60
time=31.208 ms
…
```

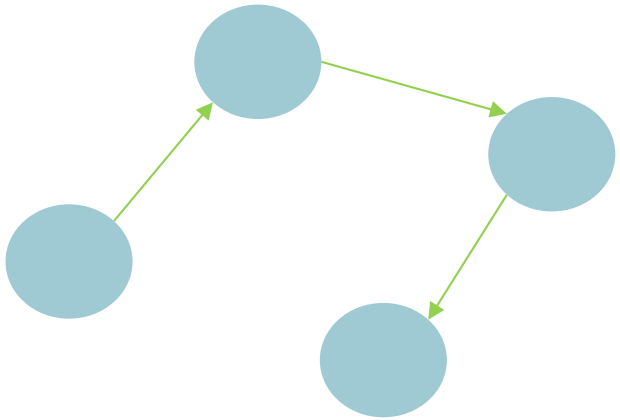
- Then run ssh using your ‘username’ e pwd (your UniTrento credentials):

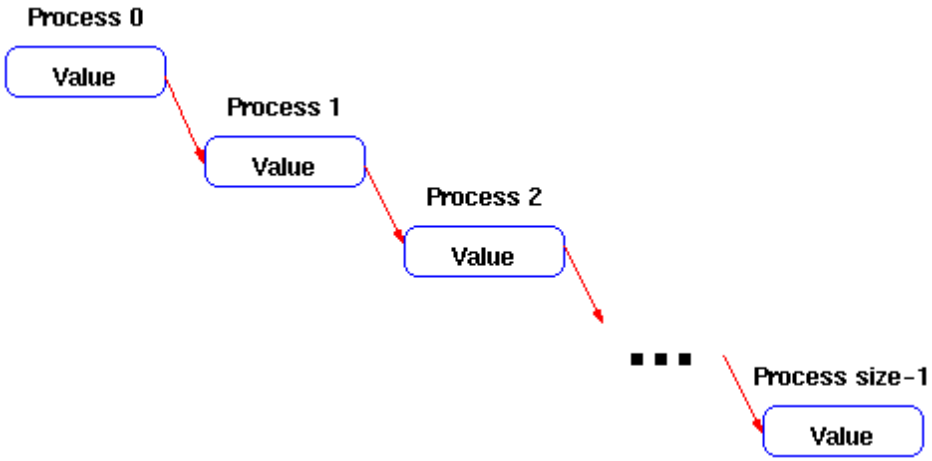
```
ssh sandro.fiore@hpc2.unitn.it
```

Communication hands-on

Implement a ring by using send and receives

Write a program that takes data from process zero and sends it to all of the other processes by sending it in a ring. That is, process i should receive the data and send it to process i+1, until the last process is reached.





Question ?

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform